How CTSM, CLM, CIME, and NEON Fit Together

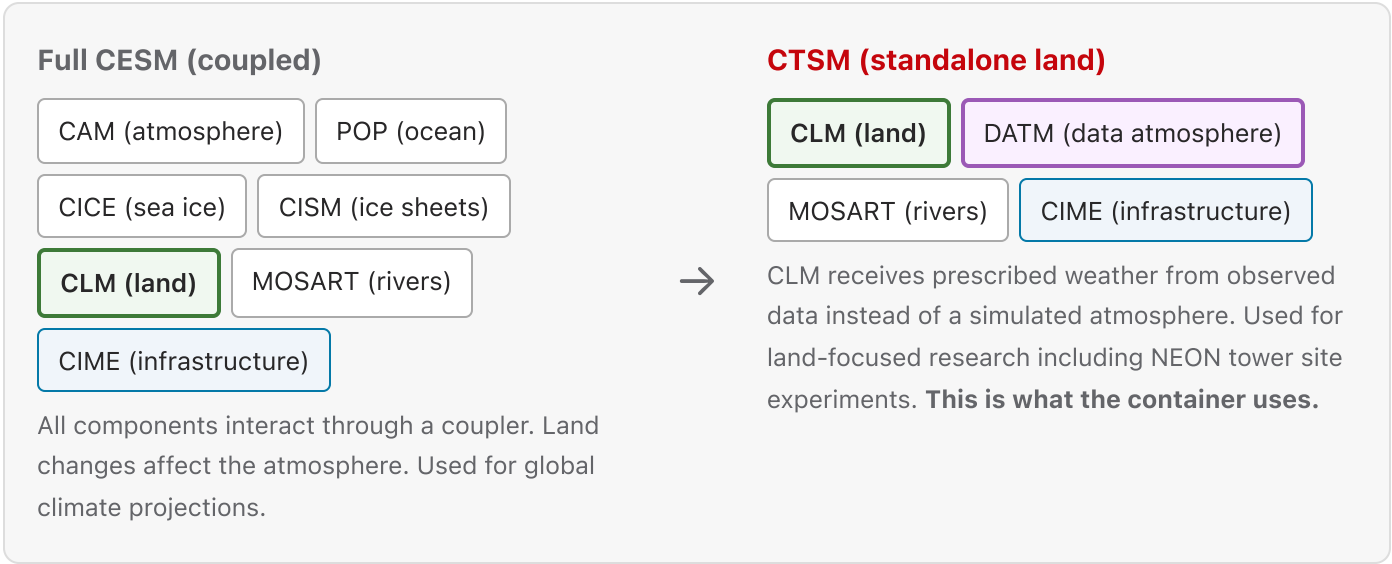
A guide to the software components in the ExSOIL container

This document explains what each piece of software in the container does, how they relate to each other, and what happens when you run a simulation at a NEON tower site. It is intended for researchers who understand the science (land surface modeling, NEON tower observations) but want a clearer picture of the software architecture.

Where CTSM Comes From: The CESM Family Tree

CESM (Community Earth System Model) is NCAR's flagship climate model. It couples separate models for the atmosphere (CAM), ocean (POP), sea ice (CICE), ice sheets (CISM), and land surface (CLM) into a single interacting Earth system. In a fully coupled CESM run, the land surface influences the atmosphere, which influences the ocean, which feeds back to the atmosphere and land. This is how researchers study large-scale climate dynamics.

CTSM (Community Terrestrial Systems Model) extracts just the land surface part -- CLM -- and gives it everything it needs to run on its own, without the atmosphere, ocean, or ice models. Instead of a simulated atmosphere, CTSM feeds CLM observed weather data (from NEON towers or reanalysis products) through a component called DATM (data atmosphere). This is much cheaper computationally and is the right tool when the research question is about the land surface itself, not about atmosphere-land-ocean interactions.



The NEON tower workflow was developed specifically for CTSM. It is not available in standard CESM releases (it will arrive when CESM 3.x ships, which is still in beta). This is why the container uses CTSM rather than CESM: it is NCAR's intended platform for running CLM at NEON sites.

The trade-off is that CTSM cannot run fully coupled experiments where land surface changes (e.g., deforestation, irrigation) feed back into the atmosphere and alter regional weather patterns. For the current project -- evaluating CLM predictions against tower observations and testing forcing perturbations -- standalone CTSM is the right tool.

What This Project Does

The ExSOIL project investigates how well CLM represents soil processes at real-world sites. The core workflow is:

1. **Run CLM at a NEON tower site** using observed atmospheric forcing data from the tower instruments (temperature, precipitation, humidity, wind, radiation).
2. **Compare CLM's predictions** (soil temperature, soil moisture, carbon fluxes) against what the tower actually measured.
3. **Evaluate model-data misfit** using statistical diagnostics (bias, RMSE, residual distributions) to quantify where and how much CLM diverges from observations.
4. **Calibrate and test sensitivity** using Kalman filter techniques and forcing perturbation experiments (e.g., "what happens to soil carbon if precipitation increases by 10%?").

This requires running the same model at many sites, with different configurations, and analyzing large volumes of NetCDF output. The container packages everything needed for this workflow so researchers can focus on the science rather than software installation.

The rest of this document explains what is inside the container and how the pieces work together.

Diagram 1: What Is in the Container

The container packages three layers of software. The outermost layer is the Docker container itself. Inside it, CTSM provides the model and build system. Alongside CTSM, the Python analysis stack provides tools for working with model output.

Docker Container (Ubuntu 24.04)

Everything packaged together. Runs on Intel/AMD and Apple Silicon.

CTSM 5.4 (Community Terrestrial Systems Model)

The standalone land modeling framework. Contains the model code and everything needed to build and run it.

CLM

Community Land Model. The Fortran code that simulates soil physics, vegetation, hydrology, carbon cycling, and snow.

CIME

Build and run infrastructure. Handles case creation, Fortran compilation, namelist generation, and job execution.

DATM (CDEPS)

Data atmosphere. Reads observed weather files and feeds them to CLM as atmospheric forcing.

MOSART

River routing. Moves water from the land surface to river channels and ultimately to the ocean boundary.

CMEPS

Coupler. Passes fields (energy, water, momentum) between components at each time step.

NEON Usermods (48 sites)

Per-site configuration: grid cell, soil properties, vegetation type, surface datasets, and namelist overrides for each NEON tower location.

Compilers

gfortran, gcc, MPICH, cmake. Build the Fortran model at runtime.

Python Analysis Stack

Tools for working with CLM output after a simulation.

xarray + netCDF4

Load and manipulate CLM history files

matplotlib + cartopy

Plotting, maps, projections

numpy + scipy + pandas

Numerical computation

JupyterLab

Notebook interface (what you see in the browser)

analytics_modules

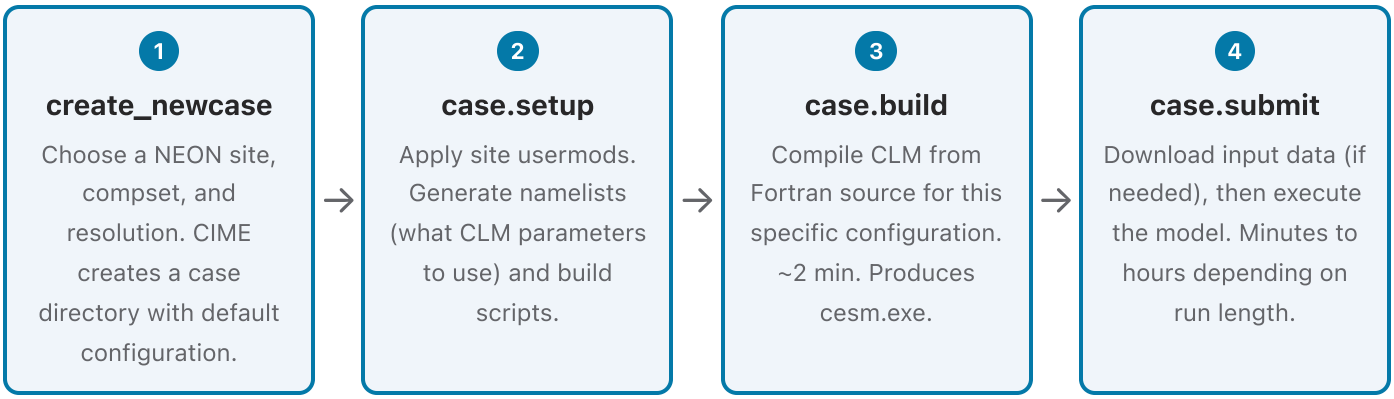
Project-specific: Kalman filter, model misfit, data access

Key relationships

Term	What it is	Analogy
CTSM	The standalone land modeling project. Contains CLM plus everything it needs to run independently.	The car
CLM	The land model itself. Fortran code that does the science: soil heat transfer, water flow, plant growth, carbon cycling.	The engine
CIME	The build and run system. Creates cases, compiles Fortran, generates namelists, manages execution.	The assembly line + ignition system
DATM	Data atmosphere. Reads weather observations from files and delivers them to CLM as if they were coming from a real atmosphere model.	A tape player feeding pre-recorded weather to the engine
A case	A configured simulation directory. One specific experiment: this site, this time period, these physics options, these parameter tweaks.	A specific trip planned with the car: destination, route, departure time
Usermods	Per-site configuration files that customize a case for a specific NEON tower. Grid cell, soil type, vegetation, surface datasets.	GPS coordinates and local road conditions for the trip

Diagram 2: What Happens When You Run a Simulation

Running a CTSM simulation at a NEON site follows a four-step pipeline managed by CIME. Data flows in (atmospheric forcing, surface parameters) and out (history files with model predictions).



Input data (flows in)

- **Surface datasets** -- soil texture, topography, vegetation type, land use for the grid cell
- **Atmospheric forcing** -- gap-filled NEON tower observations: temperature, humidity, precipitation, wind, radiation
- **Parameter files** -- CLM physics parameters (plant functional types, soil biogeochemistry constants)

Output data (flows out)

- **History files** -- NetCDF files with predicted soil temperature, moisture, carbon fluxes, energy balance, etc.
- **Restart files** -- model state snapshots for continuing a run later
- **Log files** -- timing, diagnostics, warnings

The project's `run_neon_v2` wrapper script automates steps 1-4 in a single command. When you run a notebook cell like `run_neon_v2 --neon-sites KONZ --output-root ~/CLM-NEON`, it calls CIME's commands internally, applies the KONZ usermods, and manages the build and execution.

Diagram 3: What Is a "Case"?

A case is a directory that CIME creates to hold one specific simulation experiment. You create many cases from the same CTSM installation. Each case is independent: its own configuration, its own build, its own output.

Case: KONZ.transient

Site: Konza Prairie, KS

Compset: I1PtClm60Bgc

Period: 2018-2022

`user_nl_clm` -- CLM namelist overrides

`env_run.xml` -- run length, restart options

`bld/cesm.exe` -- compiled model

`run/` -- active simulation directory

Case: HARV.transient

Site: Harvard Forest, MA

Compset: I1PtClm60Bgc

Period: 2018-2022

`user_nl_clm` -- different site params

`env_run.xml` -- same run length

`bld/cesm.exe` -- same compiled model

`run/` -- different forcing data

Case: KONZ.precip_plus10

Site: Konza Prairie, KS

Compset: I1PtClm60Bgc

Period: 2018-2022

Perturbation: precip +10%

`user_nl_clm` -- same as KONZ base

`env_run.xml` -- same run length

Uses transformed forcing data

All three cases use the same CTSM installation, the same CLM physics code, and the same Python analysis tools. They differ in which site they simulate, what forcing data they use, and

what parameter modifications they apply. This is how the project's perturbation experiments work: run a control case and a perturbed case at the same site, then compare the output.

How the Analysis Pipeline Connects

After a simulation produces history files, the Python analysis stack takes over:

Step	Tool	What it does
Load output	xarray	Opens NetCDF history files as labeled multi-dimensional arrays
Compute diagnostics	numpy, scipy, pandas	Soil temperature profiles, carbon flux time series, water balance closure
Compare to observations	analytics_modules	Model-data misfit statistics (RMSE, bias, correlation) against NEON tower measurements
Calibrate	analytics_modules	Kalman filter adjustment of simulated values toward observations
Visualize	matplotlib, cartopy	Time series plots, soil profile contours, map projections

This pipeline is what the project's notebooks implement. The **Data_Hub** focuses on data loading and visualization. The **Modeling_Hub** focuses on misfit evaluation and calibration. The **Design_Hub** focuses on running perturbation experiments and comparing control vs. modified scenarios.